任务 1：阅读代码并回答问题

1. BERT 文本分类和实体识别有什么关系？分别使用什么 Loss？

关系：

在 `model.py` 中定义的 `BertForIntentClassificationAndSlotFilling` 模型，展示了这两种任务是如何通过**共享底层特征（多任务学习）**联系在一起的。

- **共享编码器**：文本分类（意图识别）和实体识别（槽位填充）都使用同一个 BERT 模型提取特征。
- **不同的输出层**：
  - **文本分类（意图识别）**：使用的是 BERT 输出的 `pooler_output`（即 `[CLS]` 标记的特征向量）。这个向量聚合了整个句子的全局语义信息，经过一个全连接层（`sequence_classification`）映射到意图类别的数量上。
  - **实体识别（槽位填充）**：使用的是 BERT 输出的 `token_output`（即除了 `[CLS]` 和 `[SEP]` 之外的每个单词或字对应的序列特征向量）。这些向量包含了丰富的上下文局部信息，经过另一个全连接层（`token_classification`）映射到 BIO 标签类别的数量上。

使用的 Loss：

根据 `main.py` 中的代码，这两种任务分别使用了**交叉熵损失函数（CrossEntropyLoss）**。

- `seq_loss = self.criterion(seq_output, seq_label_ids)`
- `token_loss = self.criterion(active_logits, active_labels)`
- 两者都调用了初始化时的 `self.criterion = nn.CrossEntropyLoss()`。

2. 多任务训练 loss = seq_loss + token_loss 有什么坏处？如何处理训练不平衡？

直接相加的坏处：

在 `main.py` 中，总损失被简单地定义为 `loss = seq_loss + token_loss`。这种直接相加的策略通常存在以下问题：

- **收敛速度差异**：意图识别通常是一个相对简单的句子级分类任务，收敛速度较快；而槽位填充是词级别的序列标注任务，难度更大，收敛较慢。
- **梯度主导**：如果其中一个任务的 loss 值明显大于另一个（例如 token 数量多导致 token_loss 的量级远超 seq_loss），那么在反向传播时，模型的参数更新将被数值大的那个任务所主导，导致另一个任务“学不到东西”或出现过拟合。

处理训练不平衡的方法：

- **手动加权（Weighted Sum）**：为两个损失分配不同的权重参数。例如修改为 $Loss = \alpha \cdot L_{seq} + (1-\alpha) \cdot L_{token}$。通常会给较难的任务（如槽位填充）分配更大的权重。
- **动态损失加权（Dynamic Weighting）**：使用诸如不确定性加权（Uncertainty Weighting）或梯度归一化（GradNorm）的方法，让模型在训练过程中根据各个任务的当前表现自动调整权重。
- **交替训练（Alternating Training）**：在一个 Batch 或 Epoch 内，先只更新意图识别的参数，再只更新槽位填充的参数，避免梯度冲突。
- **调整学习率**：对共享层和特定任务层使用不同的学习率。例如，降低 BERT 底层的学习率，提高任务输出层的学习率。

任务 2：为信息解析写一个 LLM 提示词

采用**角色设定 + 任务约束 + 标签空间定义 + Few-Shot（少样本）示例**的结构：

```
# Role
你是一个经验丰富的对话系统 NLU（自然语言理解）引擎。你的任务是将用户的自然语言输入解析为机器可读的结构化 JSON 数据。

# Rules
1. 仔细阅读用户的输入，提取出“领域（domain）”、“意图（intent）”以及文本中包含的“实体槽位（slots）”。
2. 只能从下方提供的【待选标签空间】中选择最匹配的 domain 和 intent。
3. 抽取的实体必须是用户输入中的**原始文本**，并且必须赋予正确的实体标签类型。
4. 如果某个字段没有匹配的信息，slots 保持为空字典，不能捏造信息。
5. 必须严格以 JSON 格式输出，不要包含任何多余的解释文字或 Markdown 标记。

# Label Space
【待选领域（domain）】: music, app, radio, lottery, stock, novel, weather, match, map, website, news, message, contacts, translation, tvchannel, cinemas, cookbook, joke, riddle, telephone, video, train, poetry, flight, epg, health, email, bus, story
【待选意图（intent）】: OPEN, SEARCH, REPLAY_ALL, NUMBER_QUERY, DIAL, CLOSEPRICE_QUERY, SEND, LAUNCH, PLAY, REPLY, RISERATE_QUERY, DOWNLOAD, QUERY, LOOK_BACK, CREATE, FORWARD, DATE_QUERY, SENDCONTACTS, DEFAULT, TRANSLATION, VIEW, ROUTE, POSITION
【待选实体类型（slots）】: name, Src, Dest, startLoc_city, endLoc_city, artist, song, dishName, ingredient, category, datetime_date, datetime_time, teleOperator, content, keyword

# Examples
User: "帮我查询下从北京到天津到武汉的汽车票"
Output:
{
  "domain": "bus",
  "intent": "QUERY",
  "slots": {
    "Src": "北京",
    "Dest": "武汉"
  }
}

User: "糖醋鲤鱼怎么做啊？"
Output:
{
  "domain": "cookbook",
  "intent": "QUERY",
  "slots": {
    "dishName": "糖醋鲤鱼"
  }
}

# Task
User: "${用户输入的句子}"
```

```
Output:
```